```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f
```

```
df = pd.read_csv("/content/heart_disease_uci.csv")
df.head()
```

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg
0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy
1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy
2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy
3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal
4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy

Next steps: Generate code with df New interactive sheet

```
df.isnull().sum()
```

	0
id	0
age	0
sex	0
dataset	0
cp	0
trestbps	59
chol	30
fbs	90
restecg	2
thalch	55
exang	55
oldpeak	62
slope	309
ca	611
thal	486
num	0

dtype: int64

```
df = df.dropna()
```

```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

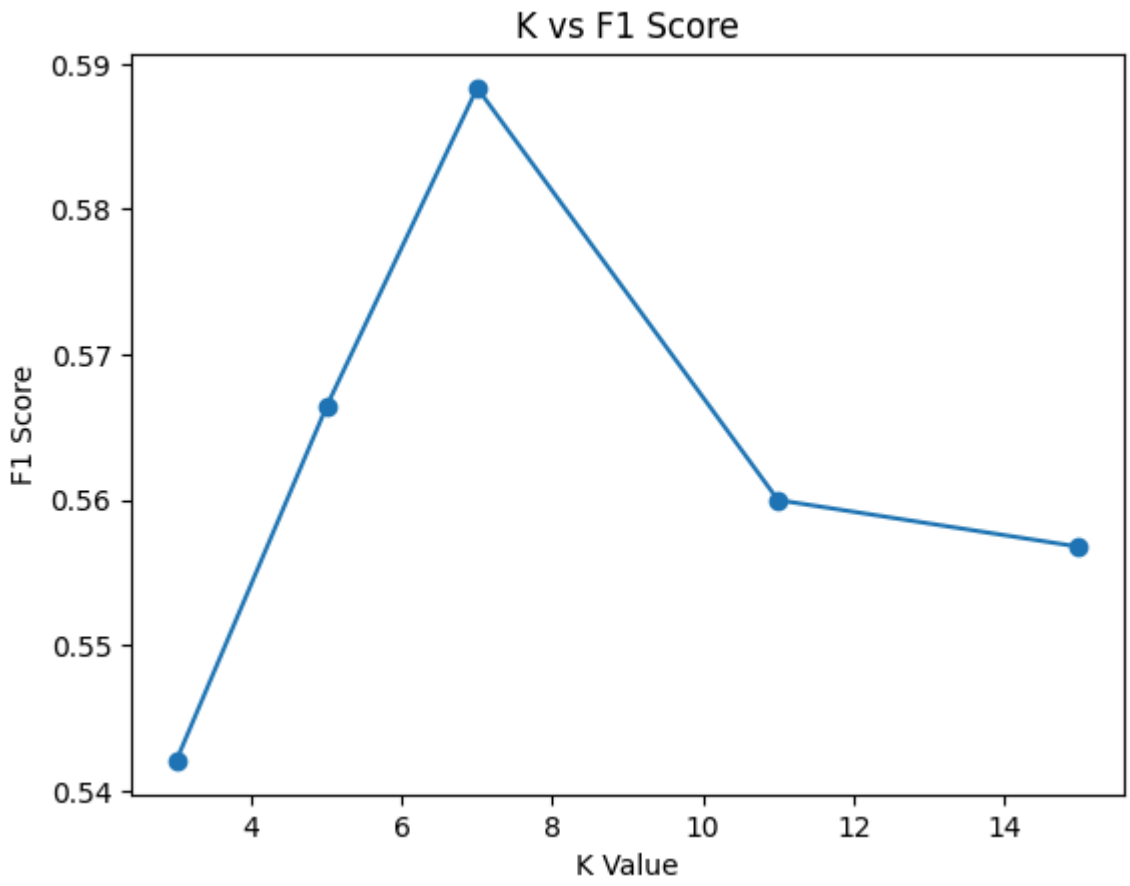
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
k_values = [3, 5, 7, 11, 15]

accuracy_list = []
precision_list = []
recall_list = []
f1_list = []
```

```
for k in k_values:

    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    accuracy_list.append(acc)
    precision_list.append(prec)
    recall_list.append(rec)
    f1_list.append(f1)

    print("K =", k)
    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1 Score:", f1)
    print("---------------------")
```

```
K = 3
Accuracy: 0.6166666666666667
Precision: 0.5028769841269842
Recall: 0.6166666666666667
F1 Score: 0.5420783132530121
---------------------
K = 5
Accuracy: 0.6166666666666667
Precision: 0.5302217915590008
Recall: 0.6166666666666667
F1 Score: 0.5664594664594665
---------------------
K = 7
Accuracy: 0.6333333333333333
Precision: 0.5538888888888889
Recall: 0.6333333333333333
F1 Score: 0.5883399209486165
---------------------
K = 11
Accuracy: 0.6166666666666667
Precision: 0.5304232804232805
Recall: 0.6166666666666667
F1 Score: 0.56
---------------------
K = 15
```

```
Accuracy: 0.6333333333333333
Precision: 0.5227304964539007
Recall: 0.6333333333333333
F1 Score: 0.5567910090865614
---------------------
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:15
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
plt.figure()
plt.plot(k_values, f1_list, marker='o')
plt.xlabel("K Value")
plt.ylabel("F1 Score")
plt.title("K vs F1 Score")
plt.show()
```